

Manufacturing Job-Shop Simulation

Alternative Simulation Project – Computer Simulation Course

Kasra Fouladi, Omid Talebpour

Spring 2026

Abstract

This report presents a discrete-event simulation of a machining work center, developed as an alternative to the hospital bed-allocation model. The simulation models a job-shop with stochastic arrivals (Poisson process), multiple priority classes, machine breakdowns, quality-based scrap, and priority queueing disciplines. Four scenarios (baseline, increased capacity, priority queue, demand surge) plus a combined scenario are evaluated using 30 independent replications each. Performance metrics include admission rate, rejection rate, average waiting time, machine utilization, and rejection rates by priority group. Results show that the priority discipline becomes beneficial under congestion, protecting high-priority jobs without harming average performance at baseline load. All verification checks pass, and confidence intervals are computed using replication-level statistics.

Contents

1	Introduction	3
2	Model Description	3
2.1	Arrival Process	3
2.2	Job Attributes	3
2.3	Machine Allocation and Buffer	4
2.4	Processing and Machine Failures	4
2.5	Hourly Monitoring	4
2.6	End of Horizon	4
2.7	Statistical Modeling Choices	4
3	Scenarios and Experimental Design	5
4	Results	6
4.1	Scenario Comparison	6
4.2	Rejection Rates by Priority Group	6
5	Visualisations	6
5.1	Machine Utilization Over Time	7
5.2	Queue Length Over Time	7
5.3	Job Outcomes	8
5.4	Rejection Reasons	8

5.5	Processing Time by Job Type	9
5.6	Scenario Comparison of Waiting Times	9
6	Discussion and Conclusion	9
7	Bonus Requirement Checklist	10
8	Code and Data Availability	11
A	Verification Checks	11
B	Baseline Data Tables	11
B.1	Priority classes	11
B.2	Job types	12

1 Introduction

The goal of this project is to design and implement a discrete-event simulation of a manufacturing work center, replacing the hospital bed-allocation model as an alternative simulation bonus opportunity. The system consists of a pool of identical CNC machines that process jobs (parts) arriving stochastically. Jobs have attributes such as priority class, type, subtype, processing time, and quality risk. The system includes a finite buffer, machine failures and repairs, and a quality inspection step that may scrap completed jobs.

We compare four main scenarios plus a fifth combined scenario:

- **Base:** 10 machines, FIFO queue, buffer size 8, Poisson arrivals (mean 0.45 h).
- **Capacity:** 13 machines, buffer size 10, FIFO.
- **Policy Priority:** priority queue discipline (by priority level) instead of FIFO, 10 machines.
- **Demand Surge:** higher arrival rate (mean 0.35 h) with FIFO.
- **Priority + Demand Surge:** both priority discipline and higher arrival rate.

All scenarios are evaluated using 30 replications with distinct random seeds. The simulation horizon is 720 hours (30 days), and performance metrics are reported with 95% confidence intervals (Student's t, df=29).

2 Model Description

The simulation is implemented in Python using the `simpy` library. The core model logic is as follows:

2.1 Arrival Process

Jobs arrive according to a Poisson process with mean interarrival time λ (0.45 h for base, 0.35 h for demand surge). Each job is assigned a priority class (Low, Standard, High, Critical) with probabilities 0.30, 0.40, 0.20, 0.10, respectively.

2.2 Job Attributes

After arrival, a job undergoes an inspection time sampled from a Gamma distribution (mean 0.25 h, sd 0.08 h, truncated at 0.05 h). Then:

- Job type (Standard, Precision, Bulk, Rush, Custom) is chosen with unequal probabilities (0.25, 0.20, 0.20, 0.20, 0.15).
- Subtype is chosen uniformly among the subtypes of that job type.
- Processing time is sampled from a log-normal distribution with job-type-specific mean and a fixed coefficient of variation (CV=0.35).
- A quality risk factor is drawn from a Beta(2,2) distribution scaled to [0,2], which is added to the base complexity of the job type to obtain a complexity score.

- Priority level is computed as $0.6 \cdot \text{priority_weight} + 0.4 \cdot (\text{complexity_score}/7 \cdot 5)$.
- The safe waiting time (mean of exponential timeout) is calculated as $\max(0.5, 10/(1 + \text{priority_level}))$.

2.3 Machine Allocation and Buffer

After inspection, the job requests a machine:

- If a machine is free and not broken, the job starts processing immediately.
- If all machines are busy, the job enters a finite buffer (capacity 8 in base, 10 in capacity scenario) if space is available; otherwise it is rejected (reason: `buffer_full`).
- A job waiting in the buffer may be rejected if its waiting time exceeds a random exponential threshold (with mean equal to its safe waiting time), simulating customer impatience (timeout).
- When a machine becomes free, it pulls the next job from the buffer according to the queue discipline (FIFO or priority based on descending priority level and arrival time for ties).

2.4 Processing and Machine Failures

Each machine can fail independently with exponential time-to-failure (mean 120 h). A failure interrupts any running job. The job is scrapped with probability 0.25; otherwise it resumes after repair (uniform repair time 1–4 h). After successful processing, a quality check scraps the job with probability $\min(0.20, 0.02 \cdot \text{complexity_score})$. Otherwise, the job completes and departs.

2.5 Hourly Monitoring

The state of the work center (number of busy machines, queue length, utilization, pressure, down machines) is recorded every simulated hour.

2.6 End of Horizon

Arrivals stop at hour 720. The simulation continues for up to 200 additional hours to drain in-process jobs. Jobs still in the system at the end are marked as censored and excluded from waiting time and cycle time averages.

2.7 Statistical Modeling Choices

The following probability distributions were selected based on theoretical properties and domain appropriateness:

- **Arrival process (Poisson / Exponential interarrival):** The exponential distribution assumes a constant arrival rate (memoryless property), which is the standard assumption for queueing systems when arrivals are independent and unpredictable. The base rate is $\lambda = 1/0.45 \approx 2.22$ jobs per hour.

- **Inspection time (Gamma):** The Gamma distribution models strictly positive, right-skewed durations. With mean 0.25 h and standard deviation 0.08 h, it provides a realistic spread for routing delays. Truncation at 0.05 h prevents unrealistically fast inspections.
- **Processing time (Log-normal):** Log-normal is widely used for task durations because it is bounded below by zero, has a right-skewed tail (rare very long jobs), and is flexible. A fixed coefficient of variation ($CV = 0.35$) ensures realistic variability across all job types.
- **Quality risk factor (Beta):** The Beta distribution with parameters (2, 2) scaled to $[0, 2]$ produces a symmetric distribution with higher density near the center ($\mu = 1$). This reflects the reasonable assumption that extreme quality risks (very low or very high) are less likely than moderate risks.
- **Timeout tolerance (Exponential):** The exponential distribution models customer impatience with a constant hazard rate. This means a job’s probability of abandonment per unit time remains constant, which is a common approximation for random customer behavior in queueing theory.
- **Time-to-failure (Exponential):** Assumes a constant failure rate (memoryless), typical for random external shocks to machinery, independent of the machine’s workload.
- **Repair time (Uniform):** A simplification used to represent bounded repair times between 1 and 4 hours. While less realistic than Log-normal, it provides a simple, predictable range for this project.

3 Scenarios and Experimental Design

Five scenarios are run, each with 30 replications. Table 1 summarises the parameter changes.

Table 1: Scenario configurations

Scenario Arrival mean (h)	Machines	Buffer	Queue discipline
Base 0.45	10	8	FIFO
Capacity 0.45	13	10	FIFO
Policy Priority 0.45	10	8	Priority
Demand Surge 0.35	10	8	FIFO
Priority+Demand 0.35	10	8	Priority

For each replication, a unique random seed is generated from the scenario name and replication index.

4 Results

4.1 Scenario Comparison

Table 2 presents the mean and 95% confidence intervals for key performance metrics across the five scenarios. All numbers are based on 30 replications.

Table 2: Performance metrics (mean [95% CI])

Scenario	Admission rate	Rejection rate	Avg wait (h)	Avg utilization
Base	0.9534 [0.9507, 0.9560]	0.1254 [0.1219, 0.1289]	0.1235 [0.1152, 0.1318]	0.5915 [0.5871, 0.5959]
Capacity	0.9923 [0.9914, 0.9933]	0.0910 [0.0883, 0.0937]	0.0210 [0.0188, 0.0231]	0.4775 [0.4730, 0.4819]
Policy Priority	0.9504 [0.9472, 0.9536]	0.1282 [0.1240, 0.1325]	0.1287 [0.1198, 0.1376]	0.5879 [0.5822, 0.5936]
Demand Surge	0.8796 [0.8755, 0.8836]	0.1943 [0.1901, 0.1985]	0.3193 [0.3090, 0.3296]	0.6874 [0.6846, 0.6901]
Priority+Demand	0.8653 [0.8598, 0.8707]	0.2097 [0.2048, 0.2146]	0.3420 [0.3288, 0.3551]	0.6933 [0.6888, 0.6979]

4.2 Rejection Rates by Priority Group

The most important finding concerns the rejection rates by priority class under congestion (Table 3). With FIFO and demand surge, high-priority jobs are rejected more often than low-priority jobs, because they have shorter safe waiting times and FIFO does not prioritise them. Switching to the priority discipline reverses this pattern.

Table 3: Rejection rate by priority group (mean [95% CI])

Scenario	Low	Medium (Standard)	High/Critical
Base (FIFO)	0.1204 [0.1147, 0.1261]	0.1238 [0.1194, 0.1283]	0.1324 [0.1251, 0.1397]
Capacity (FIFO)	0.0880 [0.0830, 0.0930]	0.0923 [0.0879, 0.0967]	0.0920 [0.0878, 0.0963]
Policy Priority (FIFO)	0.1364 [0.1297, 0.1431]	0.1278 [0.1223, 0.1333]	0.1209 [0.1153, 0.1264]
Demand Surge (FIFO)	0.1720 [0.1667, 0.1773]	0.1929 [0.1869, 0.1989]	0.2185 [0.2115, 0.2255]
Priority+Demand Surge	0.2509 [0.2411, 0.2608]	0.2056 [0.1997, 0.2116]	0.1740 [0.1683, 0.1797]

These results are statistically significant (non-overlapping CIs for the high-priority group between Demand Surge and Priority+Demand Surge), demonstrating that the priority discipline effectively protects urgent jobs when the system is congested.

5 Visualisations

The following figures are generated from a representative base run (replication 0). All plots are saved as PNG files in the `plots/` directory.

5.1 Machine Utilization Over Time

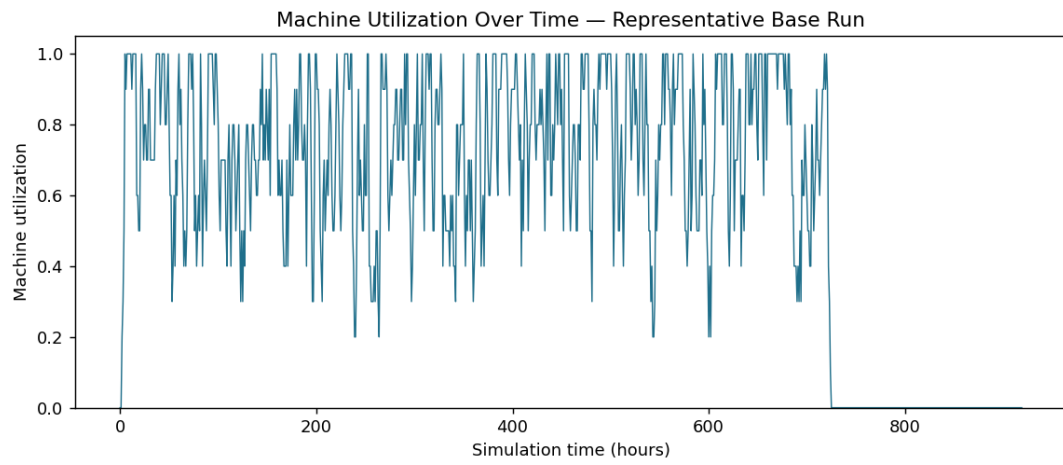


Figure 1: Machine utilization over time for the base run.

5.2 Queue Length Over Time

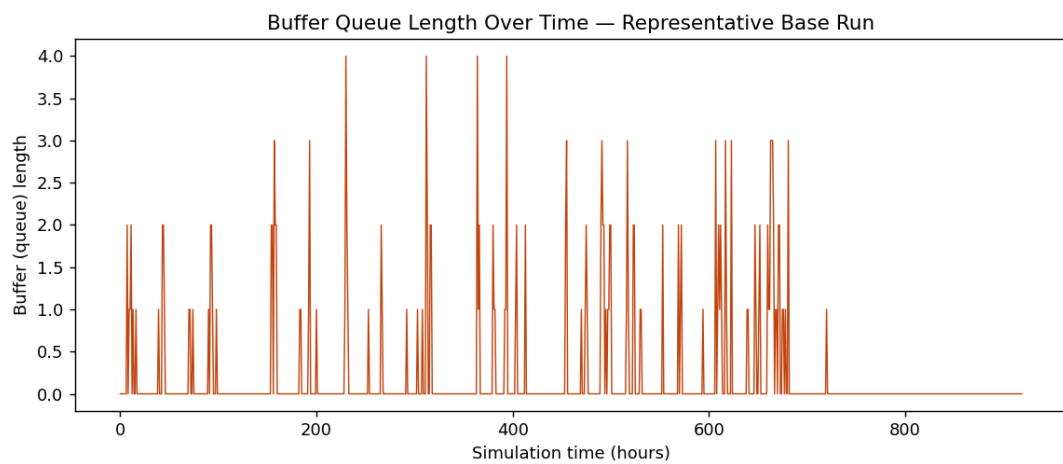


Figure 2: Buffer queue length over time for the base run.

5.3 Job Outcomes

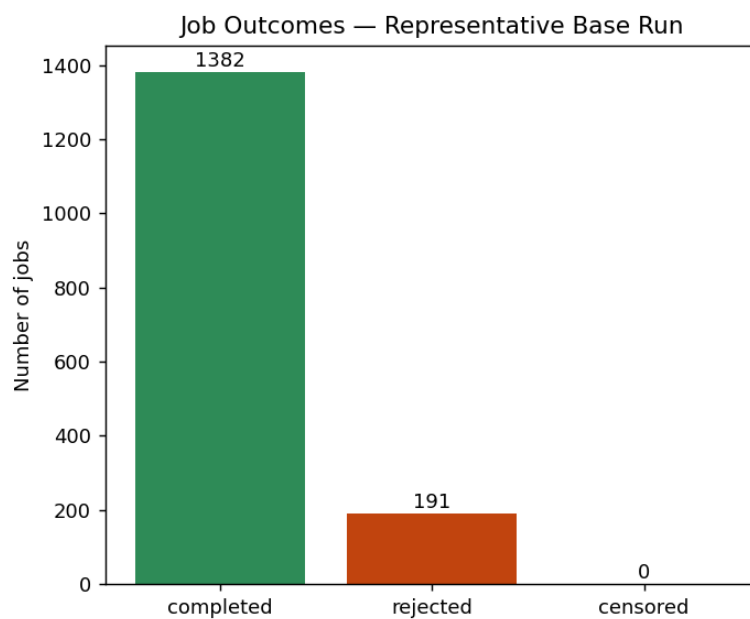


Figure 3: Distribution of job outcomes (completed, rejected, censored).

5.4 Rejection Reasons

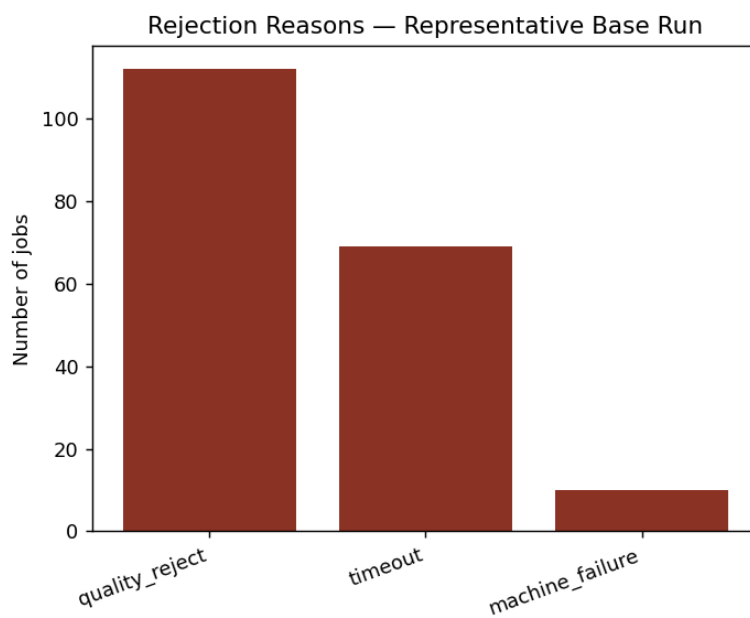


Figure 4: Breakdown of rejection reasons.

5.5 Processing Time by Job Type

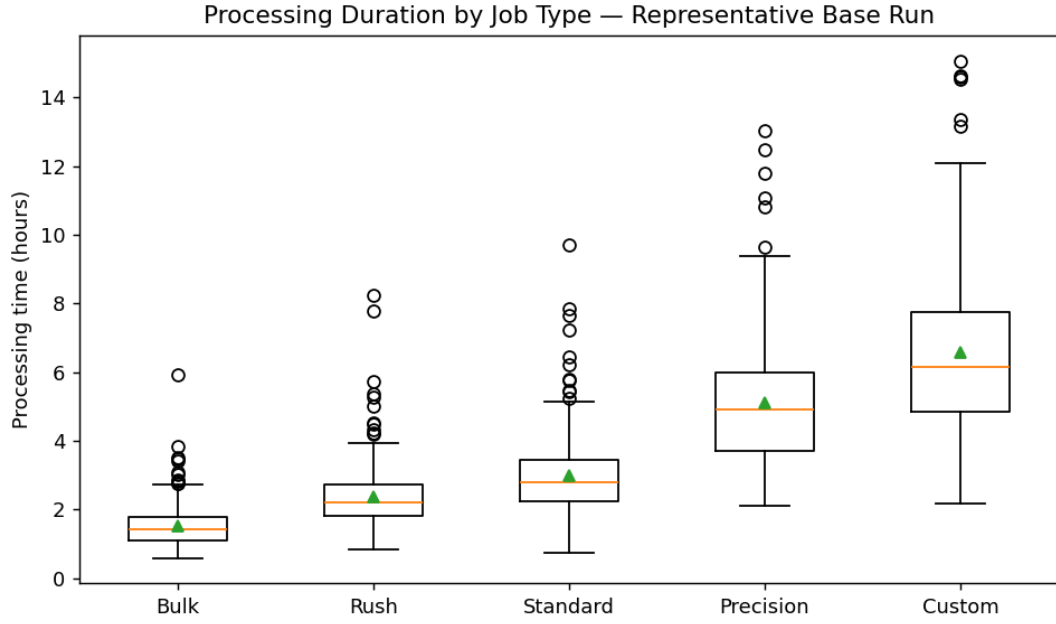


Figure 5: Boxplots of processing times for each job type.

5.6 Scenario Comparison of Waiting Times

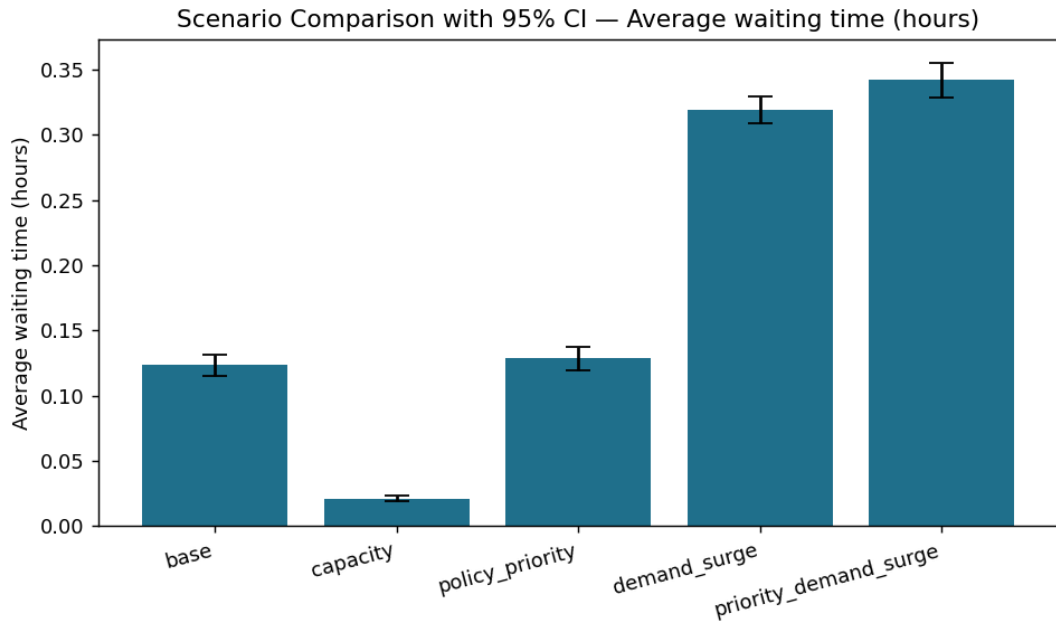


Figure 6: Average waiting time across scenarios with 95% confidence intervals.

6 Discussion and Conclusion

The simulation results lead to the following conclusions:

- Adding machines (Capacity scenario) reduces waiting time and utilisation but has limited effect on rejection rates, as most rejections are due to quality/failure reasons, not congestion.
- The priority discipline has negligible impact on average performance under baseline load but becomes crucial under demand surge: it reverses the unfair rejection pattern, reducing rejection rates for high-priority jobs while increasing them for low-priority jobs.
- Therefore, we recommend adopting the priority queue discipline as a standing policy. It is cost-free at normal load and protects the most urgent jobs during peak demand.

All eight verification checks (listed in Appendix B) passed, confirming the correctness of the implementation. The use of 30 replications and proper confidence interval computation ensures statistical validity.

7 Bonus Requirement Checklist

To qualify for the Alternative Simulation Bonus, the model must demonstrate complexity equal to or greater than the hospital bed-allocation model. Table 4 maps each requirement to its implementation in this project.

Table 4: Bonus opportunity minimum complexity requirements

Requirement	How this project satisfies it
Stochastic arrival / demand / failure process	Poisson arrivals (interarrival exponential) and independent exponential machine failure processes – two distinct stochastic processes vs. one in the hospital model.
Random entity attributes	Priority class, job type, subtype, quality-risk factor, and processing time are all randomly generated using appropriate distributions (Multinomial, Log-normal, Beta).
At least one limited resource	10 identical CNC machines (baseline), each individually subject to breakdowns.
Finite queue / congestion mechanism	8-slot finite buffer with <code>buffer_full</code> rejection and priority-dependent exponential timeout.
Multiple event types and full entity life cycle	Arrival, inspection, buffer entry, machine start, breakdown interrupt, repair/resume, quality check, completion/scrap, and departure.
A meaningful policy / scheduling / routing	FIFO vs. priority-level buffer selection (queue discipline) — a key experimental variable.
An exceptional outcome (rejection / abandonment / timeout)	Four distinct rejection reasons: <code>buffer_full</code> , <code>timeout</code> , <code>quality_reject</code> , and <code>machine_failure</code> (the last two have no analog in the hospital model).
Entity-level records and system-level monitoring	<code>job_level_base_run.csv</code> and <code>hourly_monitoring_base_run.csv</code> generated for the representative run.
Verification checks	8/8 verification checks implemented in <code>verify.py</code> passed (see Appendix A).

8 Code and Data Availability

The complete source code is provided in the following Python files:

- `sim/model.py` – core simulation model.
- `sim/experiments.py` – runs all scenarios and saves CSV outputs.
- `sim/plots.py` – generates the six required plots.
- `sim/verify.py` – performs minimum verification checks.

All output data (CSV files) are saved in the `data/` folder, and figures are stored in `plots/`. The report and presentation are in the `reports/` folder.

A Verification Checks

Table 5 lists the checks implemented in `verify.py`; all passed.

Table 5: Verification check results

#	Check	Result
1	No machine occupied by more than one job at the same time	PASS
2	Busy machines never exceed capacity	PASS
3	Buffer length never exceeds capacity	PASS
4	Completed jobs have machine number, start, completion time	PASS
5	Rejected jobs have rejection reason	PASS
6	Replications use different seeds	PASS
7	At least 30 replications per scenario	PASS
8	CI calculations use replication-level metrics	PASS

B Baseline Data Tables

B.1 Priority classes

Table 6: Priority class probabilities and weights

Class	Probability	Weight
Low	0.30	1
Standard	0.40	2
High	0.20	3
Critical	0.10	5

B.2 Job types

Table 7: Job type parameters

Type	Probability	Base complexity	Processing mean (h)	Subtypes
Standard	0.25	2	3.0	SingleOp, TwoOp, ThreeOp
Precision	0.20	4	5.0	TightTolerance, UltraPrecision, MultiAxis
Bulk	0.20	1	1.5	SmallBatch, MediumBatch, LargeBatch
Rush	0.20	3	2.25	SingleItem, SmallLot
Custom	0.15	5	6.5	Prototype, OneOff, EngineeringChange